

Computer Vision Assignment 2

SAJAD AHMAD MIR (M21MA207)

April 23, 2024

Colab link

<https://colab.research.google.com/drive/1zey06ueh9X7yqs52YRHM00yAR-Re8Ab?usp=sharing>

Introduction

K-means

K-means clustering is one of the most popular unsupervised machine learning algorithms used for clustering data. It's a method for partitioning a dataset into K distinct, non-overlapping clusters. The goal of K-means is to group similar data points together and discover underlying patterns or structures within the data.

Here's a brief introduction to how K-means works:

1. **Initialization:** The algorithm starts by randomly initializing K cluster centroids. These centroids represent the centers of the clusters.
2. **Assignment:** Each data point in the dataset is assigned to the nearest cluster centroid based on a distance metric, typically Euclidean distance. This step partitions the dataset into K clusters.
3. **Update centroids:** After all data points have been assigned to clusters, the centroid of each cluster is updated to be the mean of all data points assigned to that cluster.
4. **Repeat:** Steps 2 and 3 are repeated iteratively until convergence, i.e., until the cluster assignments and centroids no longer change significantly between iterations, or until a specified number of iterations is reached.
5. **Final clusters:** Once the algorithm converges, the final cluster centroids represent the centers of the clusters, and each data point is assigned to one of these clusters based on proximity to the centroids.

K-means aims to minimize the within-cluster sum of squares (WCSS), also known as inertia or distortion. It does this by iteratively optimizing the cluster centroids to minimize the distance between data points and their assigned centroids.

K-means is a simple and computationally efficient algorithm, making it suitable for large datasets. However, it has some limitations, such as sensitivity to the initial placement of centroids and the need to specify the number of clusters K beforehand. Despite these limitations, K-means is widely used in various fields, including data mining, image processing, and pattern recognition, for tasks such as customer segmentation, image compression, and anomaly detection.

Ratio Cut Algorithm

The Ratio Cut algorithm is a graph-based clustering technique used for partitioning data into disjoint groups or clusters. It is widely applied in tasks like image segmentation, where the goal is to divide an image into regions based on certain similarities.

The key concept of Ratio Cut is to partition a graph representing the data into clusters such that the ratio of the cut size to the volume of each cluster is minimized. This is achieved by iteratively optimizing the partitioning based on spectral clustering techniques.

Ratio Cut clustering has applications in image processing, computer vision, and pattern recognition, particularly in tasks such as image segmentation. Despite its effectiveness, it may suffer from sensitivity to noise and the need to specify the number of clusters beforehand.

1 Methodology for Image Segmentation using Ratio Cut

1.1 Graph Construction

- Represent the image as a graph, where each pixel is a node.
- Define edges between neighboring pixels, typically using 4 or 8 connectivity, depending on whether diagonal connections are considered.
- The weight of each edge is determined by the dissimilarity between the pixel intensities or color values. Lower weights indicate higher similarity.

1.2 Affinity Matrix Calculation

- Compute the affinity matrix based on the pairwise similarities between pixels. This matrix represents the strength of connections between pixels in the graph.

- Similarity measures can include color similarity, intensity difference, texture similarity, or a combination of these.

1.3 Degree Matrix Calculation

- Compute the degree matrix, which represents the sum of weights of edges connected to each pixel node.
- This matrix captures the local structural information of the graph.

1.4 Laplacian Matrix Computation

- Compute the Laplacian matrix using the affinity matrix and the degree matrix.
- The Laplacian matrix captures the overall structure and connectivity of the graph.

1.5 Eigenvalue Decomposition

- Perform eigenvalue decomposition on the Laplacian matrix to obtain the eigenvectors and eigenvalues.
- The eigenvectors corresponding to the smallest eigenvalues capture the low-frequency components of the graph.

1.6 Spectral Clustering

- Use the eigenvectors corresponding to the smallest eigenvalues for spectral clustering.
- Apply a clustering algorithm, such as K-means, to cluster the pixels based on the values of these eigenvectors.
- The number of clusters determines the segmentation regions in the image.

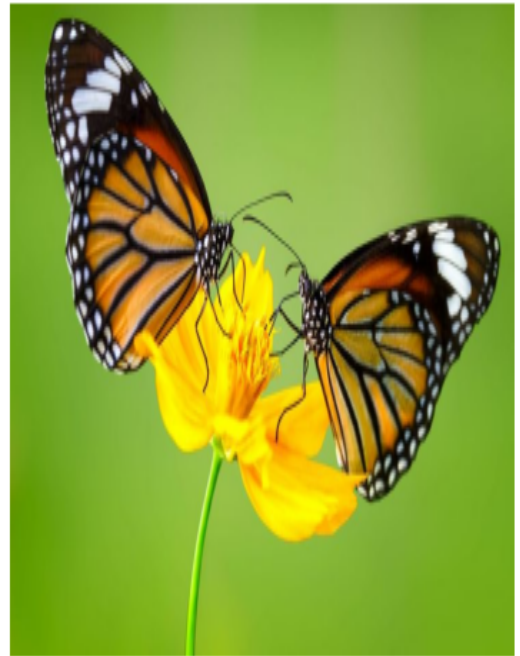
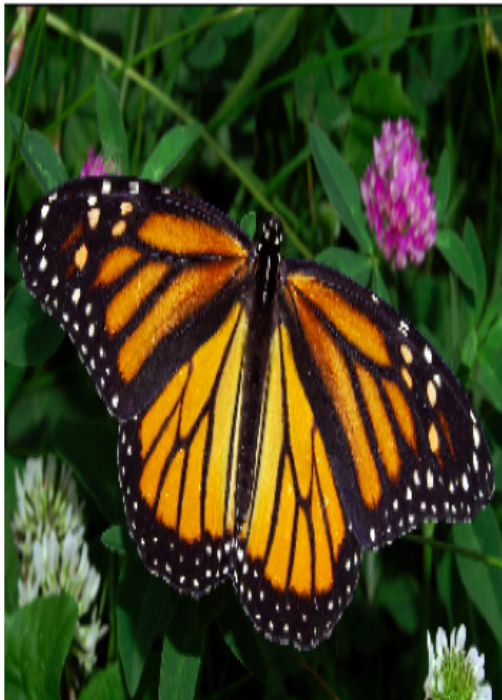
1.7 Ratio Cut Optimization

- Optimize the segmentation by minimizing the ratio cut criterion.
- The ratio cut criterion aims to minimize the ratio of the cut size to the volume of each segmented region.
- This ensures that the segmentation results in meaningful partitions with minimal dissimilarity between segments and maximum similarity within segments.

1.8 Segmentation Visualization

- Visualize the segmented regions by assigning each pixel to its corresponding cluster.
- Generate segmented images where pixels belonging to the same cluster are assigned the same label or color.

Original Images



Implementation

The provided Python code demonstrates the implementation of K-means clustering and ratio-cut clustering for image segmentation. Below is the implementation breakdown:

Importing Libraries

The code begins by importing the necessary libraries:

- `numpy` for numerical computations.
- `cv2` for image processing.
- `KMeans` from `sklearn.cluster` for K-means clustering.
- `os` for file path operations.
- `torch` and `read_image` from `torchvision.io` for image loading.
- `matplotlib.pyplot` for data visualization.
- `cv2.imshow` from `google.colab.patches` for displaying images in Google Colab.

Function Definitions

The code defines several functions:

- `load_images(image_paths)`: Loads and preprocesses images from the specified image paths.
- `resize_image(image)`: Resizes the input image to a fixed size of 64x64 pixels.
- `ratio_cut_clustering(image, k, sigma)`: Performs ratio-cut clustering on the input image with a specified number of clusters (`k`) and a Gaussian kernel parameter (`sigma`).
- `kmeans_clustering(image, k)`: Performs K-means clustering on the input image with a specified number of clusters (`k`).

Comparison Function

The code defines a `compare_segmentation(image_paths, sigmas)` function that compares the segmentation results of K-means and ratio-cut clustering for the given image paths. It iterates over different sigma values to perform ratio-cut clustering and visualizes the original image, K-means clustering (`k=3`), and ratio-cut clustering (`k=3` and `k=6`) for each sigma value.

Main Execution

The code specifies the paths to the images (`image_paths`) and a list of sigma values for ratio-cut clustering (`sigmas`). It then calls the `compare_segmentation` function with the image paths and sigma values to compare the segmentation results.

This implementation provides a framework for comparing the segmentation results of K-means clustering and ratio-cut clustering algorithms on a given set of images.

Image1

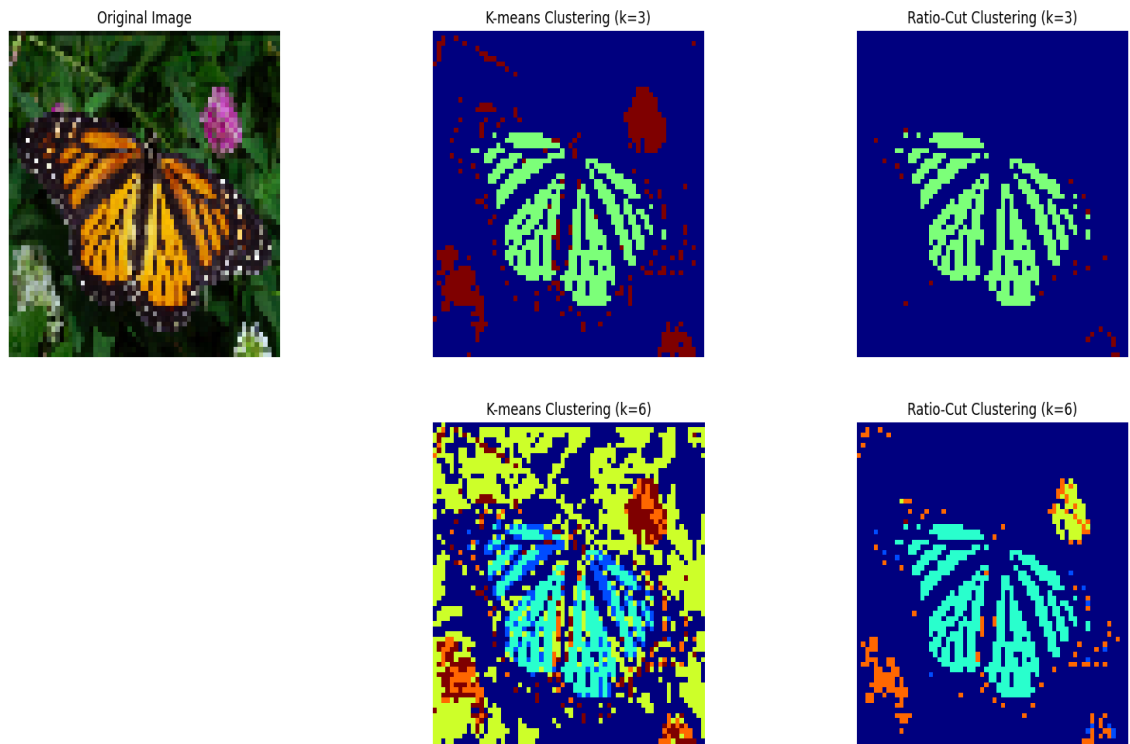
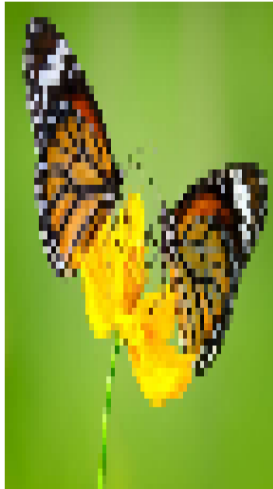
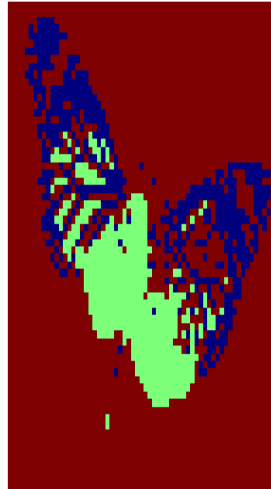


Image2

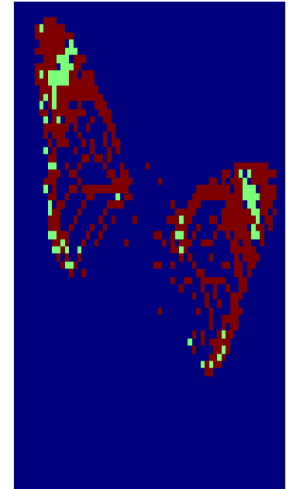
Original Image



K-means Clustering (k=3)



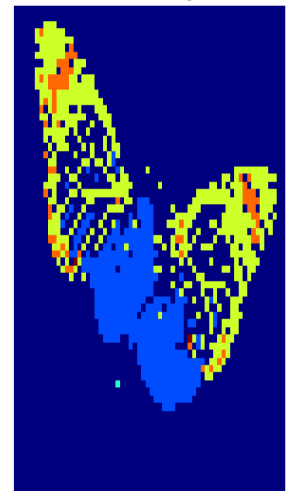
Ratio-Cut Clustering (k=3)



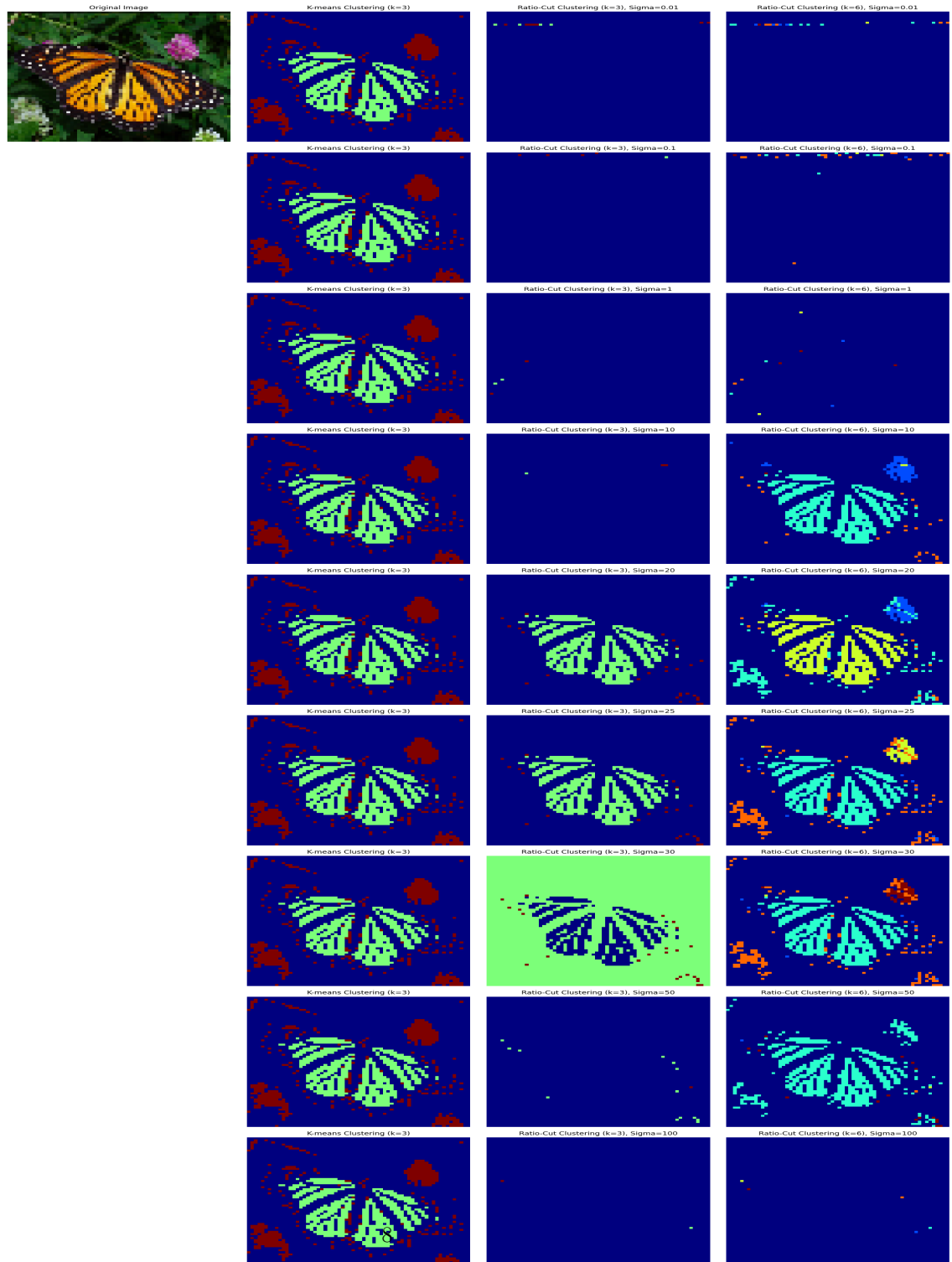
K-means Clustering (k=6)

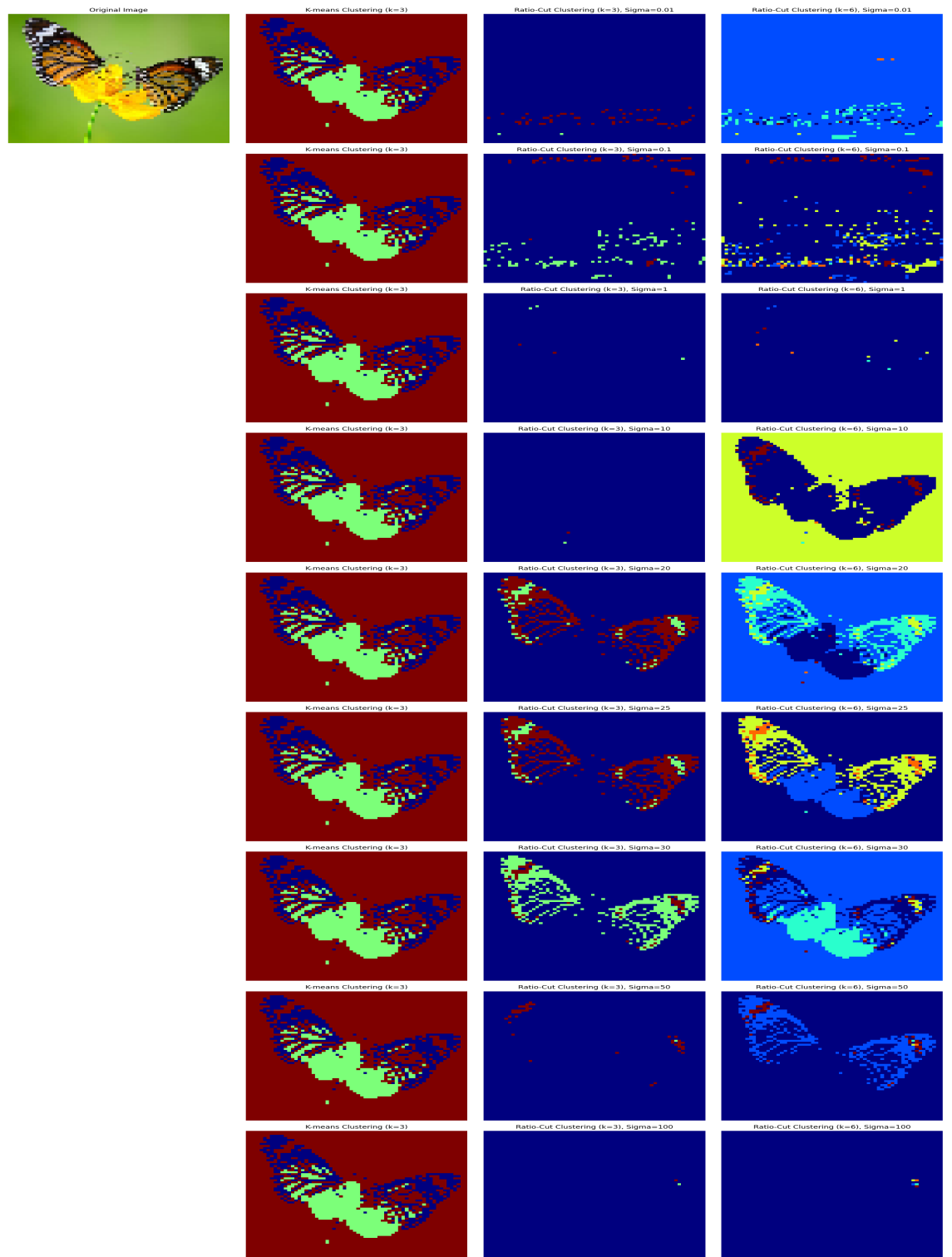


Ratio-Cut Clustering (k=6)



Tunning of Hyperparameter sigma





we check the performance of the Ratio-cut algorithm with respect to the clusters formed by K-means. After looking at different values of sigma the results are better for sigma=25